Summary Of: An Introduction To Agent-Based Modelling in R

Core Purpose of Paper

The paper demonstrates how to translate theoretical agent interactions into working R code using a Rock-Paper-Scissors game example and building off of that to spatial network simulations.

What is an Agent-Based Model (ABM)?

An Agent-Based Model is a group of computational models used to simulate the actions of independent individuals (agents) to evaluate their group effect on the overall system.

The Four Components of an ABM

Every agent-based model requires four structural components:

- Agents: Independent individuals that possess specific attributes and interact with each other or their environment. In the code, agents are represented as rows in a data.frame tracking attributes like id, strategy, and num_wins. A data.frame is similar to an excel table, a two-dimensional, labeled data structure organized into rows and columns.
- A World (Environment): This is the world where characters or participants live and move around. Simple two-player games do not need a physical space, but more complex models use a map or grid so participants can only interact with whoever is right next to them.
- A Set of Rules: Specific logic defining what agents can or must do during interactions. In the Rock-Paper-Scissors example, these rules determine strategy selection and identify which agent wins a round based on strategy hierarchy.
- An Iterative Loop: A repeated execution structure (such as a for loop) that allows agents to repeatedly choose strategies, interact, and update their states over time.

R Implementation

Instead of putting all the code in one giant block, the guide breaks the model down into smaller, reusable parts:

- Setup Function: Generates initial agent data structures to easily reset or rerun simulations.
- Behavior Functions: Custom functions like chooseStrategy assign or alter agent strategies (e.g., selecting randomly via numerical IDs).

- Interaction Rules Function: Functions like playStrategy evaluate interactions between agents, increment victory counters, or handle ties.

Hypothesis Testing through Simulation

A key purpose of ABMs is testing behavioral hypotheses through repeated experimentation.

- For example, the text sets up a simulation to test whether an agent sticking to a single fixed strategy outperforms an opponent that randomly switches strategies.
- By running 1,000 rounds across 100 repetitions and analyzing the outcomes with a two-sample t-test, the simulation demonstrates that strategy-switching does not produce a statistically significant advantage under purely random play.

Network Dynamics and Evolutionary Rules

Instead of just two players, the model puts an entire group onto a grid network where everyone only plays against their immediate neighbors. Whenever two players compete, the loser copies the winner's strategy. Over time, watching which strategies rise and fall, even when adding new moves like "Spock", shows that the population never settles on a single fixed outcome, instead, strategies continuously rise and fall in repeating waves across the network.

Summary Of: The ODD Protocol for Describing Agent-Based and Other Simulation Models

Core Purpose of Paper

This paper presents a standard way to describe agent-based simulation models. Early computer simulations were notoriously hard to explain because regular math formulas cannot easily capture complex programming rules, leaving many studies messy and difficult for other researchers to recreate. The ODD protocol fixes this problem by offering a shared, step-by-step format that anyone can read, regardless of the software they use, making it easy for others to fully understand and rebuild the exact same model.

The Structure Of ODD

ODD breaks down a model into three main sections: Overview, Design concepts, and Details, which are explained using seven steps:

Overview (O)

1. **Purpose and patterns:** Clearly states the goal of the model and the real world patterns it needs to match to prove it works realistically.
2. **Entities, state variables, and scales:** Lists the basic parts of the model, such as the individuals, the space they live in, their changing traits, and the total size and time scale covered.
3. **Process overview and scheduling:** Explains who acts, what they do, the order of events, and when the model updates its information.

Design Concepts (D)*:* Explains how the model handles core simulation ideas, such as how agents adapt, learn, interact, make choices, and respond to chance events.

Details (D)

5. **Initialization:** Shows the exact starting conditions of everything in the model right at the beginning.

6. **Input data:** Identifies any outside data fed into the simulation over time, like weather records.

7. **Submodels:** Gives the exact formulas, code rules, and step-by-step logic behind every action in the model so someone else can rebuild it completely.

Pattern-Oriented Modeling (POM)

POM uses real-world trends, like where animals live, how populations grow, or how people behave, to help build and tune the model. Researchers use these real patterns to decide what details to include, test different theories on how individuals act, and fine-tune their settings. These same patterns are stated right at the start as a clear checklist to test whether the model actually matches reality.

Explaining Design Rationale

Rather than being strictly descriptive (stating only *what* the model code contains), ODD encourages modellers to explain *why* specific design choices, parameter sources, and functional forms were chosen. Documenting design rationale improves model credibility, aids peer review, and allows other researchers to re-use proven submodels.

Skeletal AMB Chapter

1. Chapter Overview & Learning Objectives

- **Learning Goals**: Define what students will achieve (e.g., understand micro-to-macro cell dynamics, write an ABM in R/Python, analyze stochastic simulation data, and document models reproducibly).
- **Prerequisites & Suggested Background**: Brief note on required coding basics (loops, data frames) and introductory cell biology.

2. Introduction: Why Agent-Based Modeling in Oncology?

- **Comparing Modeling Approaches**:
- **When to Use ABMs**: Modeling cell-level heterogeneity, spatial competition, local microenvironmental gradients (oxygen, drug perfusion), and stochastic events (mutations, extinction).

3. The Four Pillar Ingredients of an Oncology ABM

1. **Agents**: Defining individual cell entities (e.g., tumor cells, immune killer cells, stromal cells) and their state variables (position, phenotype, proliferation capacity, mutation status, health).
2. **Environment (The World)**
3. **Interaction & Behavior Rules**: Explicit logic governing cell actions (proliferation, apoptosis, migration, drug response, or predator-prey dynamics like immune predation).
4. **Iterative Scheduling Loop**: Discrete time-stepping for updating cell states and executing interactions.

4. Hands-On Implementation Walkthrough (Code Tutorial)

- **Step 1: Data Structures**: Representing agents cleanly using data frames or matrices.
- **Step 2: Core Functions**:
    - setup_population(): Initializing spatial positions and cell types at time $t = 0$.
    - cell_behavior(): Functions for division, movement, and dying.
    - interact(): Rules for local interactions (e.g., immune cell killing tumor cell upon contact).
- **Step 3: Main Simulation Loop**: Iterating over T time steps and updating states.
- **Step 4: Spatial Visualization**: Plotting the spatial lattice/grid over time to visualize tumor morphology.

7. Standardized Model Documentation: The ODD Protocol

- **Why Standardize?**: Addressing the "replication crisis" in computational modeling.
- **The 7 Elements of ODD applied to Cancer Models**:
    - *Overview*: Purpose & Patterns, Entities & Scales, Process Overview & Scheduling.
    - *Design Concepts*: Emergence, Stochasticity, Sensing, Interactions.
    - *Details*: Initialization, Input Data, Submodel Algorithms/Equations.